

Testing & Documentation

Software Quality in Scientific Computing

16th of April 2026

Jimmy Jessen Nielsen
jjn@es.aau.dk

Dept. of Electronic Systems
Aalborg University



Testing & Documentation

Today's Agenda

- ▶ **L08 results recap** (~10 min)
 - ▶ Trajectory divergence and condition number maps — class comparison
- ▶ **Theory**: Scientific software quality (~55 min)
 - ▶ What makes code testable — extracting a computation core
 - ▶ pytest, docstrings, type hints, refactoring, reproducibility
 - ▶ Advanced: parametrize, Hypothesis, CI, mutation testing, linting, Sphinx
- ▶ **Break** (10 min)
- ▶ **Studio**: Exercises and MP3 milestones (~110 min)
 - ▶ E1: Extract and test a function from your code
 - ▶ E2: Add a docstring E3 (optional): Branch coverage report
 - ▶ M1: Test suite M2: Docstrings + type hints + ruff check
- ▶ **OpenCL environment check** (~10 min)

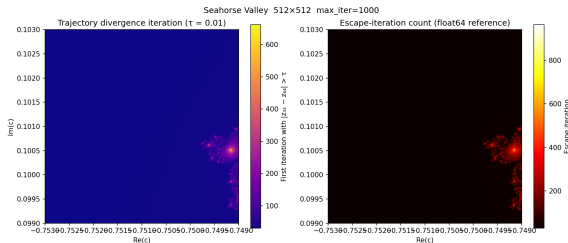
By the end of today: test suite committed, all public functions documented and type-annotated, ruff check passes — M1 and M2 done.



L08 Results

L08 Results

L08 M1 — Trajectory Divergence Map



Seahorse Valley, $\tau = 0.01$, $\text{max_iter} = 1000$

Observations:

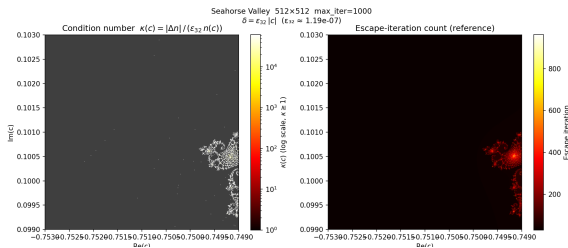
- ▶ Float32/float64 trajectories diverge *earliest* in the boundary region where escape count is highest
- ▶ Interior pixels (max_iter) and fast-escaping exterior pixels: trajectories agree — float32 sufficient there
- ▶ Early divergence $\Leftrightarrow$ high iteration count: rounding errors compound through many iterations

What did you find?

- ▶ What fraction of pixels diverged before max_iter ?
- ▶ Did your map match this spatial pattern?

L08 Results

L08 M2 — Condition Number Map



$\kappa(c) \approx |\Delta n| / (\epsilon_{32} n(c))$, LogNorm, Seahorse Valley

Observations:

- ▶ High- κ pixels trace the fractal boundary exactly — confirms $\kappa \rightarrow \infty$ as predicted by theory
- ▶ Interior pixels ($n = \text{max_iter}$): $\kappa = 0$ — the function is flat; perfectly well-conditioned
- ▶ M1 and M2 maps agree spatially: high divergence $\approx$ high κ

The connection:

The condition number explains *why* float32 fails where it does. Not random: it fails where the problem is fundamentally ill-conditioned.



Theory: Software Quality

Software Quality

When Scientific Code Goes Wrong

Two cases from the pre-class material — worth a moment before we go to tools.

Reinhart & Rogoff (2010)

- ▶ Paper: public debt $> 90\%$ GDP causes -0.1% average growth
- ▶ Cited by finance ministers; used to justify spending cuts across Europe
- ▶ 2013: PhD student replicates it — finds the Excel AVERAGE formula stops short; five countries excluded
- ▶ Corrected: $-0.1\% \rightarrow +2.2\%$
- ▶ Root cause: no test on the aggregation formula

Duke genomics (2006–2010)

- ▶ Papers claimed to predict optimal cancer chemotherapy from gene expression data
- ▶ Clinical trials opened; patients enrolled and treated
- ▶ External researchers found: sample labels shifted by one row in training data — drug sensitivities matched to the wrong cell lines
- ▶ Trials halted 2010; three papers retracted
- ▶ Root cause: no test on label alignment before training

Common thread: no automated verification, undocumented assumptions, no way to re-run and check. Your Mandelbrot codebase has the same structural gap — today you close it.



Software Quality

Scientific vs Traditional Software

Traditional software

- ▶ Purpose: provide functionality to an end-user
- ▶ Correctness: defined by a specification
- ▶ End result: a working application
- ▶ Key question: *does it do what users need?*
- ▶ Testing: against specification or user behaviour

Scientific software

- ▶ Purpose: test hypotheses, quantify results
- ▶ Correctness: defined by underlying mathematics
- ▶ End result: numerical results and plots
- ▶ Key question: *are the numbers right?*
- ▶ Testing: against known analytical solutions or cross-validated independent implementations

Why this matters for you: your Mandelbrot codebase is scientific software. A speedup claim is only credible if the result is *correct* at every step. No test $\Rightarrow$ no verification $\Rightarrow$ result is not trustworthy.

Software Quality

The Testing Gap in Scientific Computing

Empirical studies consistently show scientists do not test their code:

- ▶ Hannay et al. (2009) surveyed 1972 scientists: most are self-taught programmers; systematic testing is the least practised skill — below version control, documentation, and code review
- ▶ Segal & Morris (2008): scientists rarely release code and rarely test rigorously, making it “almost impossible to reproduce and verify published results”
- ▶ The primary reason is not laziness — it is lack of exposure: testing is not part of most scientific training

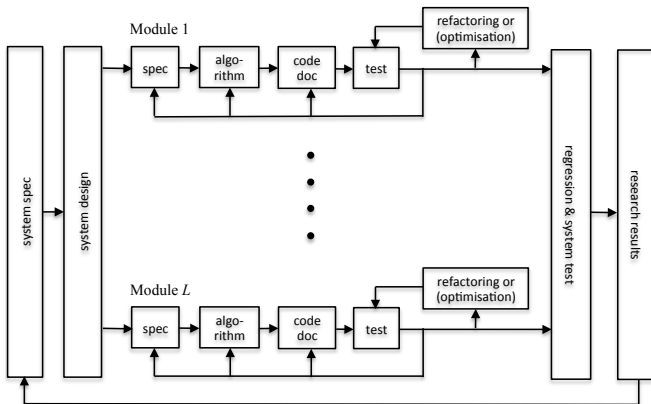
Since then: open science has raised the stakes.

- ▶ Many journals and funders now require code to be published alongside papers
- ▶ Reproducibility initiatives (e.g. Nature, PLOS) have made shared code the norm in several disciplines
- ▶ Shared code is now a scientific contribution — it should meet the same standard of rigour as the results it produces

The testing skills gap identified in 2009 has not closed. Open science raised the bar — testing is now part of what it means to do **credible computational research**.

Software Quality

Scientific Software Development Cycle



Each module iterates: spec → algorithm → **code/doc** → **test** → **refactoring**

The outer loop captures the exploratory nature of scientific work: a result changes the hypothesis, which changes the spec, which restarts the cycle. Unlike traditional software, *the target moves*.

Today: the inner module loop — code/doc, test, and refactoring.



Code & Documentation

Software Quality

Docstrings and Type Hints

Docstrings state the contract of a function. **NumPy style** is standard in scientific Python:

```
def mandelbrot_pixel(c: complex, max_iter: int) -> int:
    """Compute the escape iteration count for one complex point.

    Iterates :math:'z_{n+1} = z_n^2 + c' from :math:'z_0 = 0' until
    :math:'|z| > 2' (escape) or reaching max_iter.

    Parameters
    -----
    c : complex
        Complex coordinate to test for set membership.
    max_iter : int
        Maximum iterations; returned if trajectory does not escape.

    Returns
    -----
    int
        First iteration :math:'k' where :math:'|z_k| > 2', or 'max_iter'.
    """
    ...
```

Low barrier: a docstring is a plain string literal — no build step, no tooling.

`help(mandelbrot_pixel)` shows it immediately in any Python session.

Type hints document expected types *and* enable real-time feedback from Pylance in VSCode.

Math: $\text{\LaTeX}$ syntax is supported — `:math:'z_{n+1} = z_n^2 + c'` renders as typeset math when docs are generated. Plain text (`|z_k| > 2`) stays readable in `help()` without any tooling.

Software Quality

Advanced (Optional): Sphinx + autodoc

Generates a full HTML documentation site from your existing docstrings.

Setup:

1. `mamba install sphinx`
2. `sphinx-quickstart docs/`
3. Edit `docs/conf.py` — add at the top and to extensions:

```
import sys, os
sys.path.insert(0, os.path.abspath('.'))
extensions = ['sphinx.ext.autodoc',
'sphinx.ext.napoleon']
```

(Napoleon parses NumPy/Google docstrings)

4. `make html` from `docs/`

Result:

Your docstrings become a navigable API reference — cross-referenced, searchable, hostable on Read the Docs.

Used by NumPy, SciPy, Dask, and virtually every major scientific Python project.

Common failure points:

- ▶ Run `make html` from `docs/`, not the project root
- ▶ Omitting `sys.path` — autodoc cannot find your module
- ▶ `|z_k|` in a docstring: use `'abs(z_k)'` instead

Software Quality

Sphinx + autodoc: Result



What your docstrings can become:
A fully navigable API reference.

Section Navigation

NumPy's module structure

Array objects

The N-dimensional array
(`ndarray`)

numpy.ndarray
numpy.ndarray.flags
numpy.ndarray.shape
numpy.ndarray.strides
numpy.ndarray.ndim
numpy.ndarray.data
numpy.ndarray.size
numpy.ndarray.itemsize
numpy.ndarray.nbytes
numpy.ndarray.base
numpy.ndarray.dtype
numpy.ndarray.T
numpy.ndarray.real
numpy.ndarray.imag
numpy.ndarray.flat
numpy.ndarray.ctypes
numpy.ndarray.item
numpy.ndarray.tolist
numpy.ndarray.tobytes
numpy.ndarray.tofile
numpy.ndarray.dump
numpy.ndarray.dumps
numpy.ndarray.astype
numpy.ndarray.byteswap
numpy.ndarray.copy

🏠 > NumPy reference > Array objects > The N-dimensional array (`ndarray`)

The N-dimensional array (`ndarray`)

An `ndarray` is a (usually fixed-size) multidimensional container of items of the same type and size. The number of dimensions and items in an array is defined by its `shape`, which is a `tuple` of N non-negative integers that specify the sizes of each dimension. The type of items in the array is specified by a separate `data-type object` (`dtype`), one of which is associated with each `ndarray`.

As with other container objects in Python, the contents of an `ndarray` can be accessed and modified by [indexing or slicing](#) the array (using, for example, N integers), and via the methods and attributes of the `ndarray`.

Different `ndarrays` can share the same data, so that changes made in one `ndarray` may be visible in another. That is, an `ndarray` can be a "view" to another `ndarray`, and the data it is referring to is taken care of by the "base" `ndarray`. `ndarrays` can also be views to memory owned by Python `strings` or objects implementing the `memoryview` or `array` interfaces.

Example

Try it in your browser!

A 2-dimensional array of size 2 x 3, composed of 4-byte integer elements:

```
>>> import numpy as np
```

```
>>> x = np.array([[1, 2, 3], [4, 5, 6]], np.int32)
>>> type(x)
<class 'numpy.ndarray'>
>>> x.shape
(2, 3)
>>> x.dtype
dtype('int32')
```

☰ On this page

Constructing arrays

Indexing arrays

Internal memory layout of an
`ndarray`

Array attributes

Array methods

Arithmetic, matrix
multiplication, and comparison
operations

Special methods

Software Quality

Advanced (Mention): ADRs and Quarto

State-of-the-art tools for code documentation and scientific publishing:

Architecture Decision Records (ADRs)

- ▶ Short Markdown files documenting *why* a design decision was made
- ▶ Stored under `docs/decisions/` — one file per decision
- ▶ Template: Context → Decision → Consequences
- ▶ Low effort, high value for scientific code: “why float64 here?”, “why chunk rows and not columns?”
- ▶ Directly relevant to your performance notebook reasoning

Quarto

- ▶ Scientific publishing tool for Jupyter notebooks: renders to HTML, PDF, or slides
- ▶ Python-native; better output than “Export as PDF” from Jupyter
- ▶ `quarto render report.ipynb`
- ▶ Worth knowing for thesis work or future courses

You are not expected to use these for MP3 — return to these for future projects.

Testing

Software Quality

The assert Statement

In Python: `assert expr` raises `AssertionError` if `expr` is `False`.

That is all. pytest uses this as its testing mechanism.

```
assert 2 + 2 == 4          # passes silently
assert 2 + 2 == 5          # raises AssertionError: test fails
assert result == 100, f"expected 100, got {result}" # with message
```

A test function is just a function that asserts:

```
def test_origin():
    result = mandelbrot_pixel(0+0j, 100)
    assert result == 100
```

pytest discovers it (name starts with `test_`), calls it, and reports pass or fail.

In other languages: `assert` exists in C, C++, Java, MATLAB — but typically as a defensive check during development, disabled in production.

In Python/pytest it is the primary way to express *what correct behaviour looks like*. No special syntax needed.

Software Quality

Test Scope and Aspect

Scope (architectural level):

Level	What it tests
Unit	One function
Integration	Multiple functions/modules
System	Full pipeline, final result

Aspect (what to verify):

- ▶ **Functional correctness:** does it produce the right result?
- ▶ **Performance:** does it meet speed/usage targets? (MP1–MP3 focus)

Automation (when/how):

- ▶ **Regression tests:** existing unit tests run in CI after code changes
- ▶ **Property-based tests:** generate random inputs, verify invariants (rules that must hold for any input)

Natural progression: Unit tests → CI regression → System test

Software Quality

The Untestable Function

A typical early Mandelbrot submission:

```
def run_mandelbrot(N, max_iter):
    t0 = time.perf_counter()
    result = []
    for i in range(N):
        for j in range(N):
            z = 0j
            c = x_min + j*(x_max-x_min)/N + 1j*(y_min + i*(y_max-y_min)/N)
            for n in range(max_iter):
                if abs(z) > 2:
                    result.append(n); break
                z = z*z + c
            else:
                result.append(max_iter)
    print(f"Time: {time.perf_counter()-t0:.3f}s")
    plt.imshow(np.array(result).reshape(N, N)); plt.show()
```

What would you assert?

- ▶ No return value — nothing to assert on
- ▶ Cannot check a single pixel without running the whole grid
- ▶ Cannot run without printing and plotting
- ▶ Timing and visualisation are entangled with the computation

Software Quality

Extract → Test → Refactor

Minimal fix: pull the inner computation into a pure function.

```
def mandelbrot_pixel(c: complex, max_iter: int) -> int:
    z = 0j
    for n in range(max_iter):
        if z.real*z.real + z.imag*z.imag > 4.0:
            return n
        z = z*z + c
    return max_iter
```

Now you can assert against known values:

```
assert mandelbrot_pixel(0+0j, 100) == 100 # origin: never escapes
assert mandelbrot_pixel(5.0+0j, 100) == 1  # far outside: escapes on iter 1
```

You do not need to test everything — extract the *computation core*. Timing wrappers, plotting, and I/O do not need unit tests.

Software Quality

Three Properties of Testable Code

1. **Pure function** — same inputs always produce the same output*; no side effects (no printing, no file I/O, no global state).
 - ▶ `mandelbrot_pixel(c, max_iter)` is pure
 - ▶ `run_mandelbrot(N)` is not — it prints, plots, reads globals
2. **Single responsibility** — does one thing; describable in one sentence.
 - ▶ If you cannot describe what a function does without using “and”, split it
3. **Returns a value** — if a function only prints or plots, there is nothing to assert against.
 - ▶ Refactor to: `result = compute(...); plot(result)`
 - ▶ Test `compute`; leave `plot` untested

***Random numbers:** Seed random number generator (RNG) for reproducible results, or assert that statistical properties (e.g., mean, variance) remain stable.

Software Quality

pytest Basics

Test discovery: pytest finds `test_*.py` files and `test_*` functions automatically.

Assert: plain Python assert

```
def test_origin():  
    assert mandelbrot_pixel(0+0j, 100) == 100
```

Float / array comparison:

```
assert x == pytest.approx(1e-4, rel=1e-5)  
np.testing.assert_allclose(arr, ref, rtol=1e-6) #  
    floats  
np.testing.assert_array_equal(arr, ref)  
    # ints/exact
```

Run:

```
pytest -v  
pytest --cov=. -v
```

Coverage:

- ▶ Line coverage: which lines were executed?
- ▶ 100% line coverage $\neq$ correct code
- ▶ Coverage shows what is *untested*, not that tested code is *correct*
- ▶ Weak assertions give a false sense of safety

Install:

```
# Always use conda/mamba (not pip):  
mamba install pytest pytest-cov
```

Software Quality

LLM-Generated Test Cases

LLMs scaffold a test suite quickly — useful with eyes open.

What LLMs are good at:

- ▶ Generating boilerplate (`import pytest`, fixture setup, parametrize skeleton)
- ▶ Suggesting edge-case categories you may not have considered
- ▶ Producing many test functions quickly

The risk depends on prompt quality:

- ▶ A lazy prompt ("*write tests for this function*") produces tests that mirror the implementation — they pass by construction and catch nothing
- ▶ A test that asserts only `result is not None` is not a test
- ▶ Supplying the *spec* instead of the code avoids this — the LLM reasons about what should be true, not what the code happens to return

Best practice — two-step prompting:

1. Ask for a *test plan*: which cases to cover and why — review it before any code is written
2. Ask for the *test code* based on the approved plan



Refactoring

Software Quality

Refactoring with Tests

Definition: changing the internal structure of code without changing its externally observable behaviour. E.g., splitting a function, extracting functions to a module, renaming variables for clarity.

Rule: tests must exist before refactoring begins. Tests are the safety net — if they all pass after the change, behaviour is preserved; a failing test means you introduced a regression.

If you want to refactor code, the process is:

1. Write tests that capture current correct behaviour
2. Refactor code as required
3. Run tests — all must pass

When to refactor — objective signals:

- ▶ **KISS:** if you cannot explain a function in one sentence - split
- ▶ **Function longer than 50 lines** (IBM heuristic) - too complex
- ▶ **Cyclomatic complexity** > 10 (NIST/MathWorks):
CC = 1 + number of decision points (if/elif/for/while)
— the minimum number of test cases to cover every path.

```
def classify(x):  
    if x < 0:      # +1  
        return -1  
    elif x == 0:  # +1  
        return 0  
    else:  
        return 1  
# CC = 1 + 2 = 3
```

```
radon cc -s example.py
```

```
example.py  
F 1:0 classify - A (3)
```

Grade

A = CC 1–5 (low risk);

C = 11–15 (refactor)

Software Quality

Reproducibility and FAIR

FAIR principles (Wilson et al., 2017):

- ▶ **Findable:** code in a persistent location (Git repo)
- ▶ **Accessible:** can someone else clone and run it?
- ▶ **Interoperable:** documented package dependencies?
- ▶ **Reusable:** README, licence, usage examples?

Practical checklist:

- ▶ Each result figure: reproducible by running one script
- ▶ Environment pinned: `environment.yml`
- ▶ README: purpose, install, usage, example output
- ▶ Long runs: save intermediate results — a crash should not lose hours of computation

For your Mandelbrot project:

- ▶ Can a classmate clone your repo and reproduce your performance table from scratch?
- ▶ Is it clear which Python version and packages were used? (`environment.yml`)
- ▶ Does your README explain how to run each implementation?
- ▶ Can you re-run your L03 benchmark today and get the same number?

MP3 requires a README and `environment.yml` — these are the minimum for reproducible scientific code.



Advanced Testing

Software Quality

Advanced Testing — Tools Overview

Tool	What it does
<code>parametrize</code>	runs one test function against a list of test cases
<code>pytest-cov</code>	measures which lines and branches are exercised
Hypothesis	generates random inputs; tests structural invariants automatically
<code>mutmut</code>	injects code mutations; finds behaviours no test verifies
performance test	asserts one implementation is significantly faster than another
GitHub Actions	runs the test suite automatically on every push
<code>ruff</code> / Pylance	checks style, errors, and type consistency — without running code

Software Quality

Advanced: `pytest.mark.parametrize`

`pytest.mark.parametrize` runs one test function against multiple argument sets. `pytest` generates a separate named test for each row; failures are reported individually.

```
CASES = [  
    (input_1, expected_1),    # one tuple = one complete test scenario  
    (input_2, expected_2),  
    (input_3, expected_3),  
]  
  
@pytest.mark.parametrize("x, expected", CASES)    # must sit immediately above def  
def test_f(x, expected):  
    assert f(x) == expected
```

Bundle parameters that belong together in one scenario — `expected` depends on `input`, so they travel together as a single tuple.

New test case: add one tuple to the list. The decorator and assertion stay unchanged.

When there are multiple independent parameter axes (e.g. multiple implementations), decorators can be stacked to generate all combinations — example on the next slide.

Software Quality

Advanced: `pytest.mark.parametrize` — Mandelbrot

Apply bundling to known pixel values. **Required for M3.** Multiple implementations are an independent axis — stack a second decorator to test all combinations automatically.

```
KNOWN_CASES = [
    (0+0j, 100, 100), # origin: never escapes
    (5.0+0j, 100, 1), # far outside, escapes on iteration 1
    (-2.5+0j, 100, 1), # left tip of set
]

IMPLEMENTATIONS = [mandelbrot_pixel, mandelbrot_pixel_numba]

@pytest.mark.parametrize("impl", IMPLEMENTATIONS) # independent axis
@pytest.mark.parametrize("c, max_iter, expected", KNOWN_CASES) # bundled
def test_pixel_all(impl, c, max_iter, expected):
    assert impl(c, max_iter) == expected # 2 x 3 = 6 tests
```

Stack when axes are truly independent — every implementation should give the same answer for every input, so all combinations are meaningful.

New implementation: add to `IMPLEMENTATIONS`. New test case: add to `KNOWN_CASES`. Neither touches the other.

Software Quality

Advanced (Optional): Coverage — Overview

How thoroughly do your tests exercise the code?

Approach	What it checks	Cost	pytest support
Line	each line reached at least once	low	<code>pytest-cov</code>
Branch	each True/False outcome of every decision	low	<code>--cov-branch</code>
Path	every combination of branch outcomes	exponential	—
Hypothesis	many random paths through the code (probabilistic)	moderate	<code>hypothesis</code>

Branch coverage does not test all *combinations* of branches — only that each individual branch was taken at least once. Path coverage grows as 2^n for n decisions and is impractical. Hypothesis is the closest practical approximation.

Software Quality

Advanced (Optional): Branch Coverage

`--cov-branch` requires both outcomes of every decision to be exercised — line coverage only checks that a line was reached at all.

```
pytest --cov=. --cov-branch --cov-report=term-missing #"." is current directory
```

Cases where line coverage is blind but branch coverage catches it:

- ▶ **if with no else body, always True in tests** — all lines hit, but the False path never taken
- ▶ **Loop that always executes at least once** — the “never entered” path (empty list or `range(0)`) has no uncovered lines, but is never tested

Branch coverage does not test all *combinations* of branches — only that each individual branch was taken at least once. For n independent decisions there are 2^n path combinations; branch coverage requires only $2n$ cases (one True and one False per decision).

Software Quality

Advanced (Optional): Hypothesis

Property-based testing: instead of specific test cases, describe a rule that must hold for *any* input — Hypothesis generates random inputs to try to break it. When a failure is found it automatically simplifies to the smallest value that still triggers the bug.

Strategies describe how to generate inputs; parameters constrain the range:

```
from hypothesis import given, settings
from hypothesis.strategies import integers

@given(integers())           # generates random integers
def test_abs_non_negative(x):
    assert abs(x) >= 0       # must hold for ANY integer
```

- ▶ Built-in strategies: `integers()`, `floats(...)`, `lists(...)`, `complex_numbers(...)`
- ▶ Strategy parameters (`min_value`, `max_value`, `allow_nan`, ...) restrict to inputs that make sense for your function — choice of bounds requires thought (see next slide)
- ▶ `@settings(max_examples=200)`: how many random inputs to try (default: 100)

`mamba install hypothesis`

Software Quality

Advanced (Optional): Hypothesis — Mandelbrot

Two invariants for `mandelbrot_pixel` — stated as properties first, then as code:

Invariant 1: The result is always in $[0, \text{max_iter}]$ for any complex input.

```
# Draw random points with |c| <= 3 (covers both inside and outside the set)
@given(complex_numbers(max_magnitude=3.0, allow_nan=False, allow_infinity=False))
@settings(max_examples=200)
def test_result_in_range(c):
    assert 0 <= mandelbrot_pixel(c, 100) <= 100
```

Invariant 2: Any point well outside the set ($|c| > 2$) must escape.

```
# Draw random points far outside the set (|c| between 3 and 10)
@given(complex_numbers(min_magnitude=3.0, max_magnitude=10.0,
                       allow_nan=False, allow_infinity=False))
def test_outside_set_escapes(c):
    assert mandelbrot_pixel(c, 100) < 100
```

`allow_nan=False, allow_infinity=False`: exclude NaN and infinity as permitted inputs.

Software Quality

Advanced (Optional): Mutation Testing

Coverage shows which lines were *reached*. Mutation testing asks: *if the code contained a subtle bug, would your tests catch it?*

mutmut tries one small change at a time — replacing operators ($> \rightarrow \geq$, $< \rightarrow \leq$), changing constants ($4.0 \rightarrow 3.0$), swapping booleans ($\text{and} \rightarrow \text{or}$) — and runs the test suite after each. A **surviving mutant** is a change no test detected.

```
mamba install mutmut
mutmut run          # inject mutations and run tests
mutmut results      # list surviving mutants
```

Example: $> 4.0 \rightarrow \geq 4.0$ in the escape condition.

For $c = 0+2j$: $z_1 = 2j$, $|z_1|^2 = 4.0$ exactly.

Original: $4.0 > 4.0$ is False $\rightarrow$ continues $\rightarrow$ returns 2.

Mutant: $4.0 \geq 4.0$ is True $\rightarrow$ escapes $\rightarrow$ returns 1.

```
assert mandelbrot_pixel(0+2j, 100) == 2 # catches it: mutant returns 1, assertion fails
assert mandelbrot_pixel(0+2j, 100) < 100 # misses it: mutant returns 1, assertion passes
```

Not every surviving mutant is a real bug — some mutations produce equivalent behaviour.

Software Quality

Advanced (Optional): Performance Regression Test

Assert that one implementation is significantly faster than another on a controlled grid.
Formulate assertions in terms of execution time difference or speed-up:

```
import time, numpy as np

def time_once(fn, *args):
    t0 = time.perf_counter()
    fn(*args)
    return time.perf_counter() - t0

def test_numpy_faster_than_naive():
    N, MAX_ITER = 64, 100    # small grid: low variance, fast to run
    t_naive = time_once(mandelbrot_naive, N, MAX_ITER)
    t_numpy = time_once(mandelbrot_numpy, N, MAX_ITER)
    assert t_numpy < t_naive / 5, (
        f"NumPy ({t_numpy:.4f}s) not 5x faster than naive ({t_naive:.4f}s)"
    )
```

Keep performance tests on small grids where timing variance is low. If the test is occasionally flaky, increase the threshold or guard with `pytest.mark.skipif`.

Software Quality

Advanced (Optional): CI with GitHub Actions

Run your test suite automatically on every push. Create `.github/workflows/test.yml`:

```
name: Tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.11" }
      - run: pip install numpy pytest pytest-cov
      - run: pytest --cov=. -v
```

Easiest way to create this file: on GitHub.com, go to your repo → Actions tab → “set up a workflow yourself” — paste the YAML above and commit directly in the browser. Avoids any token permission issues.

Once it is green: intentionally break a test (e.g. change `== 100` to `== 99` for the origin), push, and watch the run fail — then fix it and see it go green again. This is the feedback loop CI gives you on every future change.

Software Quality

Static Code Analysis — Linters

Linters check code **without running it** for style violations, undefined names, unused variables, and common error patterns. Complementary to Pylance: Pylance handles type checking; linters check style, imports, and error patterns.

Recommended: `ruff` — fast, modern linter (replaces `flake8`, `isort`, and more):

```
mamba install ruff
ruff check mandelbrot.py # style, errors, suspicious patterns
ruff format mandelbrot.py # auto-format (like black)
```

In VSCode — works continuously as you type; results appear in the Problems tab:

- ▶ **Pylance** (bundled with the Python extension) — real-time type checking. Enable: Settings → search “type checking mode” → set to basic or standard (strict exists but generates noise from incomplete numpy stubs)
- ▶ **Ruff extension** — real-time linting for style and error patterns

In Spyder: Source → “Run static code analysis” (`pyflakes` + `pycodestyle`) — similar feedback shown inline in the editor.

Studio

Overview

Exercises (no hand-in):

- ▶ **E1:** Find one hard-to-test function in your code; extract the computation core; write one test for it
- ▶ **E2:** Add a NumPy-style docstring to one Mandelbrot function
- ▶ **E3 (optional):** Branch coverage report (`--cov-branch`)

MP3 Milestones:

- ▶ **M1:** `test_mandelbrot.py` with ≥ 3 tests, at least one `@pytest.mark.parametrize`; record pass count + coverage % in performance notebook → commit
- ▶ **M2:** Docstrings + type hints on one implementation (all public functions) + `ruff check` passes → commit

Advanced (optional):

- ▶ **E4:** Hypothesis **E5:** Performance regression **E6:** GitHub Actions CI
- ▶ **E7:** Sphinx **E8:** mutmut mutation testing

Carry-over from L08: finish M1 and M2 before starting M1/M2 if not yet committed.

Studio Session

E1: Extract and Test

Instructions

Goal: find a hard-to-test function in your own code; extract the computation core as a pure function; write one test.

Step 1 — identify the problem:

Does a function mix computation with timing, printing, or plotting? That is the one to fix. The “Untestable Function” slide is the reference.

Step 2 — extract:

Pull the inner pixel computation into a standalone function that takes `c` and `max_iter` and returns an `int`.

Step 3 — test:

- ▶ If your code is already well-structured, extract `mandelbrot_pixel` from your naive or Numba implementation and test it
- ▶ Choose a test value you can justify mathematically — not one you read off a plot

✓ **Done?** Run `pytest -v` and confirm the test passes → discuss what “hard to test” looked like in your code.

E2: Add a Docstring

Instructions

Goal: add a NumPy-style docstring to one Mandelbrot function.

Choose a function that:

- ▶ Does computation (not just timing or plotting)
- ▶ Has at least two parameters with non-obvious types or semantics
- ▶ Returns a meaningful value

Sections to include (NumPy style):

1. One-line summary sentence (ends with a period)
2. Parameters — name, type, and description for each parameter
3. Returns — type and description of the return value
4. Examples (optional but useful)

Test your docstring: read it aloud. Does it tell a new reader everything they need to call the function correctly? Does it state what the return value means, not just its type?

✓ **Done?** Discuss with a neighbour — compare style choices and what you chose to document.

E3 (Optional): Branch Coverage Report

Instructions

Goal: generate a branch coverage report and identify untested branches.

```
# Install if needed:
mamba install pytest pytest-cov

# Run with branch coverage:
pytest --cov=. --cov-branch --cov-report=term-missing -v
```

What to look for:

- ▶ Lines listed under “missing” — never executed by any test
- ▶ Branch arrows such as 3->5 — the if whose else path was never taken
- ▶ In `mandelbrot_pixel`: both sides of the escape condition must be covered — a pixel that escapes and a pixel that does not

✓ **Done?** Identify one untested branch; write a test that covers it → discuss with a neighbour.

M1: Test Suite

Testing Numba, Multiprocessing, and Dask

Each parallelism layer adds complexity — test the algorithm, not the framework.

▶ Numba @njit:

- ▶ The compiled function is behaviourally identical to the Python version
- ▶ Test the Python version first; the Numba version passes the same tests
- ▶ Warm-up call needed before *timing*, not before correctness tests

▶ Multiprocessing Pool:

- ▶ Test the worker function (`mandelbrot_pixel`) in isolation — it is pure
- ▶ Integration test: the assembled grid must match the serial result on a small grid
- ▶ Do not test Pool machinery — that is Python's responsibility

▶ Dask:

- ▶ Test the underlying compute function, not the scheduler
- ▶ Integration: `future = client.submit(f, arg); assert client.gather(future) == expected`

▶ Performance benchmarks:

- ▶ Do not assert on absolute timing — hardware-dependent and non-deterministic
- ▶ Performance regression tests use *relative* assertions on a small grid: `assert numpy_time < 0.1 * naive_time`

M1: Test Suite

Instructions

Goal: create `test_mandelbrot.py` with a meaningful test suite. **Requirements:**

- ▶ ≥ 3 individual test functions on core computation functions (not on timing, plotting, or I/O)
- ▶ At least one test using `@pytest.mark.parametrize` — across multiple inputs *or* across multiple implementations
- ▶ Assertions must verify *correctness*, not just “runs without crashing”

What are trustworthy test values?

- ▶ **Analytically provable** — derived from the definition, no implementation needed. Choose points whose behaviour you can prove from the mathematics alone and state why.
- ▶ **Cross-validation** — naive loop as oracle; test that NumPy/Numba/multiprocessing agree on a small grid (32×32)
- ▶ **Float32 note** — cross-validate float32 implementations against each other, not against float64 (boundary pixels will differ)

Run and record:

1. `pytest -v` — note pass count
2. `pytest --cov=. -v` — note coverage %

✓ **Done?** Record pass count and coverage % in your performance notebook (MP3) → commit.

M2: Docstrings and Type Hints

Instructions

Goal: fully document one version of your Mandelbrot code — docstrings and type hints on every public function; verify with `ruff` check.

Step 1 — pick one implementation and add to every public function:

- ▶ A docstring in NumPy style (one-line summary + Parameters + Returns)
- ▶ Type hints on all parameters and return values

Step 2 — run ruff:

```
mamba install ruff  
ruff check your_mandelbrot.py  # fix all reported issues before committing
```

Type hints also power Pylance:

- ▶ In VSCode: Settings → Settings → search “type checking mode” → set to basic
- ▶ Pylance uses your annotations to flag type mismatches in real time (Problems tab)

✓ **Done?** `ruff` check reports 0 errors → commit.

Testing & Documentation

Advanced Exercises — Overview

For students who finish M1/M2 quickly. Each maps to a theory slide.

Label	Tool	Goal
E4	Hypothesis	Write property tests for two structural invariants of <code>mandelbrot_pixel</code>
E5	pytest timing	Add a performance regression test asserting NumPy is significantly faster than naive
E6	GitHub Actions	Set up a workflow that runs your test suite automatically on every push
E7	Sphinx	Generate HTML documentation from your docstrings
E8	mutmut	Run mutation testing; identify surviving mutants; add tests to kill them

```
mamba install hypothesis  mamba install sphinx  mamba install mutmut
```



OpenCL Environment Check

Testing & Documentation

GPU Environment Check — Before L10

L10 is on Wednesday 22 April. Check today — so you will have the possibility to get it working before next session.

Step 1: Install PyOpenCL

```
mamba install pyopencl
```

Step 2: Run the platform test

```
python ../materials/L10/code/basic_test.py
```

Expected output:

```
Platform: Apple / NVIDIA / AMD / Intel    (depends on your hardware)  
Device:   GPU / CPU (PoCL fallback)  
Test kernel: PASSED
```

- ▶ **GPU detected** — ready for L10
- ▶ **CPU via PoCL** — acceptable fallback; performance lower but code works
- ▶ **Import error / no platform** — raise your hand now

✓ **Done?** Note platform and device found → continue with M1/M2 if not yet finished.